

FMS Scheduling Optimization: Technical Plan

Step-by-step experimental validation, Ulusoy benchmark alignment, and statistical reporting workflow.

Primary Directive: Maintain a disciplined single-variable experimental progression. We must resolve the foundational benchmark authenticity before running additional scale experiments, changing machine learning architectures, or finalizing documentation.

PHASE 1

Build a Ulusoy-Faithful Benchmark Generator

P0 • BLOCKING PREREQUISITE

Root Issue: `scheduler.py` currently synthesizes purely random operations, whereas the report claims Ulusoy et al. as the benchmark ground truth. All downstream conclusions depend on real data authenticity.

Parameter Dimension	Current (Broken) Pipeline	Ulusoy et al. Ground Truth Spec (pp. 7–8)
Hardware & Layout	Arbitrary matrix setup	4 machines, 2 AGVs, 3 physical layouts. Travel times = real distances ÷ 40 m/min speed + 0.5 min load/unload each.
Job & Part Pool	Random machine assignment	4 job sets drawn from 2 part pools. Deterministic routings (e.g., {4-2-3} visits machine 4 → 2 → 3).
Processing Times	Uniform random $p_{ij} \in [5, 30]$ min	Uniform random integers $p_{ij} \in [3, 15]$ minutes.
Ratio Band ($\bar{t}/\bar{p}$)	Unenforced / Ignored	Target travel-to-processing ratio within [0.05, 0.20] across problem instances. Reject and resample violators.

- ☐ Rewrite `generate_problem()` to draw parts from pool 1 and pool 2 with strict routing sequences.
- ☐ Encode exact layout travel-time matrices for the 3 Ulusoy physical configurations.
- ☐ Sample $p_{ij} \sim [3, 15]$ and implement instance rejection/resampling to strictly enforce the **[0.05, 0.20]** $\bar{t}/\bar{p}$ ratio.
- ☐ Establish principled scale generation: Use job sets 2/4 (15–20 parts per job) instead of ad-hoc incrementing of `n_jobs`.

Strategic Value: Fixes data authenticity so our benchmark claim is literal truth, providing defensible large-scale problem sets for subsequent evaluation.

PHASE 2

Re-run Existing Pipeline at Dual Scale

P0 • CORE EXPERIMENT

Single-Variable Discipline: Keep `ml_layer.py`, `hill_climbing.py`, `genetic_algorithm.py`, and `main.py` ablation logic frozen. Only the input instance generator changes.

- ☐ **Scale Regime A (Small ≤ 12 jobs):** Re-evaluate on faithful Ulusoy instances to determine whether earlier null results ($p = 0.28$, $p = 0.83$) were artifacts of synthetic noise.
- ☐ **Scale Regime B (Large Scale):** Evaluate job sets 2 and 4 (5 jobs $\times$ 15–20 parts $\approx$ 45–100+ operations) to directly validate the hypothesis: *"Does ML prioritization yield advantages at scale?"*
- ☐ Directly extract and compare the 4 outcome distributions: Small-Baseline vs Small-ML, and Large-Baseline vs Large-ML.

Milestone Checkpoint: Isolates scale as the sole independent variable before evaluating alternative neural architectures.

PHASE 3

Extend Statistical Architecture for Two-Scale Regimes

P1 • RIGOROUS ANALYSIS

`anova_test.py` currently aggregates all 40 problems into an omnibus block, obscuring potential regime-specific performance.

- ☐ Decouple tests: Report paired Wilcoxon signed-rank tests (`GHCA` vs `ML-GHCA` ; `Heuristic` vs `ML-GHCA`) separately for small vs. large regimes.
- ☐ Add effect size calculation (matched-pairs rank-biserial correlation r) alongside p-values to decouple practical effect from sample size.
- ☐ Extend honest-failure reporting logic to flag null or negative gains per scale regime rather than across pooled aggregates.

PHASE 4

Outstanding Code-Quality & Pipeline Refactoring

P1 • TECH DEBT RESOLUTION

- ☐ Delete `tempCodeRunnerFile.py` to remove redundant and buggy `calculate_oct` implementations.
- ☐ Eliminate dead computation in `ml_layer.py`: delete unused `hill_climbing(sorted_ops)` call inside `ml_ghca()`.
- ☐ Conduct hyperparameter sensitivity test for cost weights: re-test $0.7 / 0.3$ vs $0.5 / 0.5$ and $0.9 / 0.1$ to verify method ranking stability.
- ☐ Update plotting schema (`plot_results.py`, `plot_convergence.py`): replace fixed `ops_per_job` assumptions with `Num_Parts` and `Num_Operations` to account for variable part routings.

PHASE 5

Isolated CNN / MLP Model Capacity Ablation P2 • OPTIONAL ARCHITECTURE EXPLORATION

Pre-condition: Execute only after Phase 2 delivers a finalized, stable linear baseline.

- ☐ Hold dataset, GA, HC, and objective functions constant; swap only the regressor (`LinearRegression` $\rightarrow$ per-operation MLP / 1D-CNN).
- ☐ If utilizing Conv1D, provide mathematical and domain justification for spatial/sequential locality over 6 scalar features; structure inputs with contiguous job groupings.
- ☐ Report as an independent secondary ablation table answering: *"Does model capacity affect schedule quality?"*

Rewrite Academic Report to Align with Empirical Findings

- ☐ **Conclusion:** Strip references to 10 job sets / 20 iterative runs; summarize actual experimental outcomes.
- ☐ **Abstract & Methodology:** Document true Ulusoy generator parameters, citations, paired Wilcoxon methodology, and 4-tier ablation scheme.
- ☐ **Results Section:** Transparently disclose null/marginal findings at small scale alongside large-scale empirical shifts.
- ☐ **Section 8 (Key Findings):** Eliminate claims of "statistically significant improvement" wherever unsupported by Wilcoxon tests.

Repository Hygiene & Viva Reproducibility Polish

- ☐ Create explicit `requirements.txt` (`numpy` , `scipy` , `matplotlib` , `scikit-learn` , `torch` conditional).
- ☐ Untrack and gitignore build artifacts (`git rm --cached tempCodeRunnerFile.py`).
- ☐ Provide 3-command reproduction guide in `README.md` (data generation → statistical testing → figure output).
- ☐ Document all heuristic hyperparameters (`GENERATIONS=200` , `POPULATION_SIZE=20` , `MUTATION_RATE=0.3` , `iterations=1000`) with rationales.